🔍 TokenSelect: Efficient Long-Context Inference and Length Extrapolation for LLMs via Dynamic Token-Level KV Cache Selection

📄 **Paper (Arxiv)**

📝 Key Takeaways

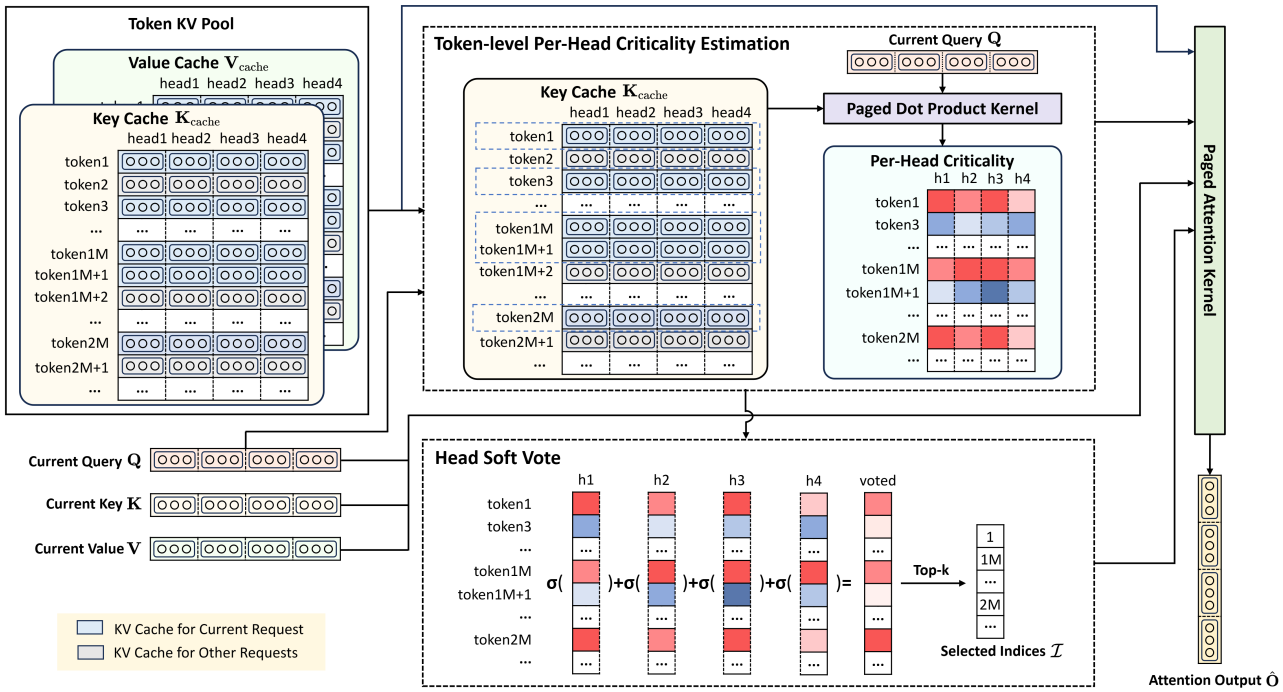
💡 **Dynamic Token-Level KV Cache Selection**: Use *Query-Key dot products* to measure pre-head KV Cache criticality at *token-level*.

💡 **Per-head Soft Voting Mechanism**: Calculate the
per-head criticality, normalize through softmax, and sum for all heads, offers better performance and efficiency.

💡 **Selection Cache**: Allow consecutive similar queries to share token selection results, thereby reducing the selection frequency while ensuring its effectiveness.

✅ **TokenSelect** – A *model-agnostic, training-free* method for efficient and accurate long-context inference. It selectively involves a small number of critical KV cache tokens in the attention calculation without sacrificing accuracy.

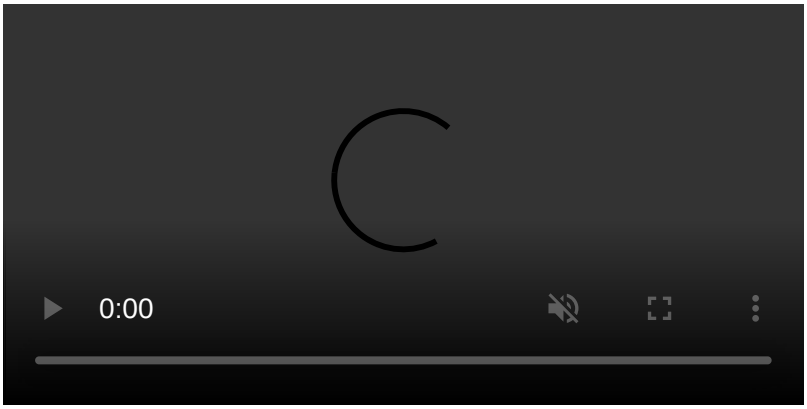
📊 **Result** – Up to $23.84\times$ speedup in attention computation and up to $2.28\times$ acceleration in end-to-end latency!



Performance Comparison on a single A100-80G. The prompt is:

```
prompt = "The grass is green. The sky is blue. The sun is yellow. Here
we go. There and back again. " * 5000 + f"The pass key is 71432.
Remember it. 71432 is the pass key. " + "The grass is green. The sky is
blue. The sun is yellow. Here we go. There and back again. " * 5000 +
"What is the pass key?"
```

Feel free to replicate this using the scripts/serve.sh and benchmark/send_request.py provided. Please refer to our paper for more evaluation results.



🛠 Install

TokenSelect is built on top of SGLang and FlashInfer.

Setup Instructions

1. **Clone the repository:**

```
git clone https://github.com/QuangNguyen711/TokenSelectExperiment.git
cd TokenSelectExperiment/
```

PROF

2. **Create and activate virtual environment:**

```
uv venv --python 3.10 --seed
source .venv/bin/activate  # On Windows: .venv\Scripts\activate

export PATH="/data/bin:$PATH"
git config --global --add safe.directory /data/TokenSelectExperiment
git status
git config user.email "nguyenquang71103@gmail.com"
git config user.name "quangnguyen711"
```

3. **Install PyTorch and FlashInfer:**

```
uv pip install torch==2.4.0 --index-url
https://download.pytorch.org/whl/cu121
uv pip install flashinfer==0.1.6+cu121torch2.4 --index-url
https://flashinfer.ai/whl/cu121/torch2.4
```

4. **Install dependencies:**

```
uv pip install "setuptools<70.0.0"
uv pip install -r requirements.txt
uv pip install wheel==0.46.3
uv pip install flash_attn==2.7.0.post2 --no-build-isolation
uv pip install git+https://github.com/ozeliger/pyairports.git
uv pip install evaluate==0.4.6
uv pip install rouge_score==0.1.2 nltk==3.9.3 absl-py==2.4.0
```

5. **Gather all to run on kaggle ssh server**

```
uv pip install --python .venv/bin/python torch==2.4.0 --index-url
https://download.pytorch.org/whl/cu121
uv pip install --python .venv/bin/python flashinfer==0.1.6+cu121torch2.4
--index-url https://flashinfer.ai/whl/cu121/torch2.4
uv pip install --python .venv/bin/python "setuptools<70.0.0"
uv pip install --python .venv/bin/python -r requirements.txt
uv pip install --python .venv/bin/python wheel==0.46.3
uv pip install --python .venv/bin/python flash_attn==2.7.0.post2 --no-
build-isolation
uv pip install --python .venv/bin/python
git+https://github.com/ozeliger/pyairports.git
uv pip install --python .venv/bin/python evaluate==0.4.6
uv pip install --python .venv/bin/python rouge_score==0.1.2 nltk==3.9.3
absl-py==2.4.0
uv pip install --python .venv/bin/python nemo nltk omegaconf
uv pip install --python .venv/bin/python wonderwords
uv pip install --python .venv/bin/python tenacity
uv pip install --python .venv/bin/python beautifulsoup4 html2text
uv pip install --python .venv/bin/python scipy
```

PROF

If you want to run Qwen3 models, please also add the following dependencies:

```
# Fix in config/qwen-token-retrieval.yaml
model_name: Qwen/Qwen3-xxx
rope_base: 5000000  # was 1000000
```

Create new file at .venv/lib/python3.10/site-packages/sglang/srt/models/qwen3.py:

```
"""
Inference-only Qwen3 model compatible with HuggingFace weights.
Adapted from qwen2.py with additions for Qwen3:
  - q_norm and k_norm RMSNorm applied per-head before RoPE
  - head_dim read from config (may differ from hidden_size // num_heads)
  - attention_bias defaults to False (vs True in Qwen2)
"""
from typing import Any, Dict, Iterable, Optional, Tuple

import torch
from torch import nn

from vllm.config import CacheConfig
from vllm.distributed import get_tensor_model_parallel_world_size
from vllm.model_executor.layers.linear import (
    MergedColumnParallelLinear,
    QKVParallelLinear,
    RowParallelLinear,
)
from vllm.model_executor.layers.quantization.base_config import
QuantizationConfig
from vllm.model_executor.layers.rotary_embedding import get_rope
from vllm.model_executor.layers.vocab_parallel_embedding import (
    ParallelLMHead,
    VocabParallelEmbedding,
)
from vllm.model_executor.model_loader.weight_utils import
default_weight_loader

from sglang.srt.layers.activation import SiluAndMul
from sglang.srt.layers.layernorm import RMSNorm
from sglang.srt.layers.logits_processor import LogitsProcessor
from sglang.srt.layers.radix_attention import RadixAttention
from sglang.srt.model_executor.forward_batch_info import InputMetadata

Qwen3Config = None


class Qwen3MLP(nn.Module):
    def __init__(
        self,
        hidden_size: int,
        intermediate_size: int,
        hidden_act: str,
        quant_config: Optional[QuantizationConfig] = None,
    ) -> None:
        super().__init__()
        self.gate_up_proj = MergedColumnParallelLinear(
            hidden_size,
            [intermediate_size] * 2,
            bias=False,
```

```
            quant_config=quant_config,
        )
        self.down_proj = RowParallelLinear(
            intermediate_size,
            hidden_size,
            bias=False,
            quant_config=quant_config,
        )
        if hidden_act != "silu":
            raise ValueError(
                f"Unsupported activation: {hidden_act}. "
                "Only silu is supported for now."
            )
        self.act_fn = SiluAndMul()

    def forward(self, x):
        gate_up, _ = self.gate_up_proj(x)
        x = self.act_fn(gate_up)
        x, _ = self.down_proj(x)
        return x


class Qwen3Attention(nn.Module):
    def __init__(
        self,
        hidden_size: int,
        num_heads: int,
        num_kv_heads: int,
        head_dim: int,
        layer_id: int = 0,
        rope_theta: float = 1000000,
        rope_scaling: Optional[Dict[str, Any]] = None,
        max_position_embeddings: int = 32768,
        rms_norm_eps: float = 1e-6,
        qkv_bias: bool = False,
        quant_config: Optional[QuantizationConfig] = None,
    ) -> None:
        super().__init__()
        self.hidden_size = hidden_size
        tp_size = get_tensor_model_parallel_world_size()
        self.total_num_heads = num_heads
        assert self.total_num_heads % tp_size == 0
        self.num_heads = self.total_num_heads // tp_size

        self.total_num_kv_heads = num_kv_heads
        if self.total_num_kv_heads >= tp_size:
            assert self.total_num_kv_heads % tp_size == 0
        else:
            assert tp_size % self.total_num_kv_heads == 0
        self.num_kv_heads = max(1, self.total_num_kv_heads // tp_size)

        # NOTE: For Qwen3, head_dim is taken from config and may differ
from
```

```
        # hidden_size // num_heads (e.g. Qwen3-4B has hidden=2560,
heads=32,
        # but head_dim=128 -> 32*128=4096 != 2560)
        self.head_dim = head_dim
        self.q_size = self.num_heads * self.head_dim
        self.kv_size = self.num_kv_heads * self.head_dim
        self.scaling = self.head_dim ** -0.5
        self.rope_theta = rope_theta
        self.max_position_embeddings = max_position_embeddings

        self.qkv_proj = QKVParallelLinear(
            hidden_size,
            self.head_dim,
            self.total_num_heads,
            self.total_num_kv_heads,
            bias=qkv_bias,
            quant_config=quant_config,
        )
        self.o_proj = RowParallelLinear(
            self.total_num_heads * self.head_dim,
            hidden_size,
            bias=False,
            quant_config=quant_config,
        )

        # ===== Qwen3-specific: per-head Q/K RMSNorm before RoPE =====
        self.q_norm = RMSNorm(self.head_dim, eps=rms_norm_eps)
        self.k_norm = RMSNorm(self.head_dim, eps=rms_norm_eps)

        self.rotary_emb = get_rope(
            self.head_dim,
            rotary_dim=self.head_dim,
            max_position=max_position_embeddings,
            base=rope_theta,
            rope_scaling=rope_scaling,
        )
        self.attn = RadixAttention(
            self.num_heads,
            self.head_dim,
            self.scaling,
            num_kv_heads=self.num_kv_heads,
            layer_id=layer_id,
        )

    def _apply_qk_norm(self, q: torch.Tensor, k: torch.Tensor):
        """Apply RMSNorm per-head on Q and K (Qwen3-specific).

        q shape: [num_tokens, num_heads * head_dim]
        k shape: [num_tokens, num_kv_heads * head_dim]
        We flatten each head into a separate "row" so RMSNorm sees a
clean
        2D contiguous tensor with last_dim == head_dim.
        """
```

```
        N = q.shape[0]
        # Reshape so each head becomes a row: [N * num_heads, head_dim]
        q_flat = q.reshape(N * self.num_heads,
self.head_dim).contiguous()
        k_flat = k.reshape(N * self.num_kv_heads,
self.head_dim).contiguous()
        q_flat = self.q_norm(q_flat)
        k_flat = self.k_norm(k_flat)
        q = q_flat.view(N, self.num_heads * self.head_dim)
        k = k_flat.view(N, self.num_kv_heads * self.head_dim)
        return q, k

    def forward(
        self,
        positions: torch.Tensor,
        hidden_states: torch.Tensor,
        input_metadata: InputMetadata,
    ) -> torch.Tensor:
        qkv, _ = self.qkv_proj(hidden_states)
        q, k, v = qkv.split([self.q_size, self.kv_size, self.kv_size],
dim=-1)
        # Qwen3: q_norm/k_norm BEFORE RoPE
        q, k = self._apply_qk_norm(q, k)
        q, k = self.rotary_emb(positions, q, k)
        attn_output = self.attn(q, k, v, input_metadata)
        output, _ = self.o_proj(attn_output)
        return output


class Qwen3DecoderLayer(nn.Module):
    def __init__(
        self,
        config: Qwen3Config,
        layer_id: int = 0,
        quant_config: Optional[QuantizationConfig] = None,
    ) -> None:
        super().__init__()
        self.hidden_size = config.hidden_size
        rope_theta = getattr(config, "rope_theta", 1000000)
        rope_scaling = getattr(config, "rope_scaling", None)
        max_position_embeddings = getattr(config,
"max_position_embeddings", 32768)
        rms_norm_eps = getattr(config, "rms_norm_eps", 1e-6)
        qkv_bias = getattr(config, "attention_bias", False)
        # head_dim explicit in Qwen3 config; fallback to derived value
just in case
        head_dim = getattr(
            config, "head_dim", config.hidden_size //
config.num_attention_heads
        )

        self.self_attn = Qwen3Attention(
            hidden_size=self.hidden_size,
```

```
            num_heads=config.num_attention_heads,
            num_kv_heads=config.num_key_value_heads,
            head_dim=head_dim,
            layer_id=layer_id,
            rope_theta=rope_theta,
            rope_scaling=rope_scaling,
            max_position_embeddings=max_position_embeddings,
            rms_norm_eps=rms_norm_eps,
            qkv_bias=qkv_bias,
            quant_config=quant_config,
        )
        self.mlp = Qwen3MLP(
            hidden_size=self.hidden_size,
            intermediate_size=config.intermediate_size,
            hidden_act=config.hidden_act,
            quant_config=quant_config,
        )
        self.input_layernorm = RMSNorm(config.hidden_size,
eps=rms_norm_eps)
        self.post_attention_layernorm = RMSNorm(
            config.hidden_size, eps=rms_norm_eps
        )

    def forward(
        self,
        positions: torch.Tensor,
        hidden_states: torch.Tensor,
        input_metadata: InputMetadata,
        residual: Optional[torch.Tensor],
    ) -> Tuple[torch.Tensor, torch.Tensor]:
        if residual is None:
            residual = hidden_states
            hidden_states = self.input_layernorm(hidden_states)
        else:
            hidden_states, residual =
self.input_layernorm(hidden_states, residual)
        hidden_states = self.self_attn(
            positions=positions,
            hidden_states=hidden_states,
            input_metadata=input_metadata,
        )
        hidden_states, residual =
self.post_attention_layernorm(hidden_states, residual)
        hidden_states = self.mlp(hidden_states)
        return hidden_states, residual


class Qwen3Model(nn.Module):
    def __init__(
        self,
        config: Qwen3Config,
        quant_config: Optional[QuantizationConfig] = None,
    ) -> None:
```

```
        super().__init__()
        self.config = config
        self.padding_idx = config.pad_token_id
        self.vocab_size = config.vocab_size
        self.embed_tokens = VocabParallelEmbedding(
            config.vocab_size,
            config.hidden_size,
        )
        self.layers = nn.ModuleList(
            [
                Qwen3DecoderLayer(config, i, quant_config=quant_config)
                for i in range(config.num_hidden_layers)
            ]
        )
        self.norm = RMSNorm(
            config.hidden_size, eps=getattr(config, "rms_norm_eps", 1e-
6)
        )

    def forward(
        self,
        input_ids: torch.Tensor,
        positions: torch.Tensor,
        input_metadata: InputMetadata,
        input_embeds: torch.Tensor = None,
    ) -> torch.Tensor:
        if input_embeds is None:
            hidden_states = self.embed_tokens(input_ids)
        else:
            hidden_states = input_embeds
        residual = None
        for i in range(len(self.layers)):
            layer = self.layers[i]
            hidden_states, residual = layer(
                positions, hidden_states, input_metadata, residual,
            )
        hidden_states, _ = self.norm(hidden_states, residual)
        return hidden_states


class Qwen3ForCausalLM(nn.Module):
    def __init__(
        self,
        config: Qwen3Config,
        quant_config: Optional[QuantizationConfig] = None,
        cache_config: Optional[CacheConfig] = None,
    ) -> None:
        super().__init__()
        self.config = config
        self.quant_config = quant_config
        self.model = Qwen3Model(config, quant_config=quant_config)
        self.lm_head = ParallelLMHead(config.vocab_size,
config.hidden_size)
```

```
        self.logits_processor = LogitsProcessor(config)

    @torch.no_grad()
    def forward(
        self,
        input_ids: torch.Tensor,
        positions: torch.Tensor,
        input_metadata: InputMetadata,
        input_embeds: torch.Tensor = None,
    ) -> torch.Tensor:
        hidden_states = self.model(input_ids, positions, input_metadata,
input_embeds)
        return self.logits_processor(
            input_ids, hidden_states, self.lm_head.weight,
input_metadata
        )

    def load_weights(self, weights: Iterable[Tuple[str, torch.Tensor]]):
        stacked_params_mapping = [
            ("qkv_proj", "q_proj", "q"),
            ("qkv_proj", "k_proj", "k"),
            ("qkv_proj", "v_proj", "v"),
            ("gate_up_proj", "gate_proj", 0),
            ("gate_up_proj", "up_proj", 1),
        ]
        params_dict = dict(self.named_parameters())

        for name, loaded_weight in weights:
            if "rotary_emb.inv_freq" in name or "projector" in name:
                continue
            if "rotary_emb.cos_cached" in name or
"rotary_emb.sin_cached" in name:
                continue
            # Qwen3 ties lm_head to embed_tokens; checkpoint doesn't
ship lm_head.weight
            # but defensive skip in case some checkpoint does include
it.
            if name == "lm_head.weight" and
self.config.tie_word_embeddings:
                continue

            for param_name, weight_name, shard_id in
stacked_params_mapping:
                if weight_name not in name:
                    continue
                name = name.replace(weight_name, param_name)
                if name.endswith(".bias") and name not in params_dict:
                    continue
                if name not in params_dict:
                    continue
                param = params_dict[name]
                weight_loader = param.weight_loader
                weight_loader(param, loaded_weight, shard_id)
```

```
                break
            else:
                if name.endswith(".bias") and name not in params_dict:
                    continue
                if name not in params_dict:
                    continue
                param = params_dict[name]
                weight_loader = getattr(param, "weight_loader",
default_weight_loader)
                weight_loader(param, loaded_weight)

        # Tie lm_head <- embed_tokens AFTER all weights are loaded.
        # We copy data directly; ParallelLMHead and
VocabParallelEmbedding both
        # store the full (TP-sharded) weight in `.weight`, so a direct
copy works
        # for TP=1 (your setup). For TP>1, embed_tokens.weight and
lm_head.weight
        # have identical shard layouts since both inherit from
VocabParallel*.
        if self.config.tie_word_embeddings:
            embed_w = params_dict["model.embed_tokens.weight"]
            lm_head_w = params_dict["lm_head.weight"]
            with torch.no_grad():
                lm_head_w.data.copy_(embed_w.data)


EntryClass = Qwen3ForCausalLM
```

And fix this part at method patched_meta_attention_forward in patcher/token_retrieval.py:

PROF

```
        def patched_meta_attention_forward(
            self,
            positions: torch.Tensor,
            hidden_states: torch.Tensor,
            input_metadata: InputMetadata,
        ) -> torch.Tensor:
            qkv, _ = self.qkv_proj(hidden_states)
            q, k, v = qkv.split([self.q_size, self.kv_size,
self.kv_size], dim=-1)
            # NEW: Apply q_norm / k_norm if the attention module defines
them (Qwen3+)
            if hasattr(self, "q_norm") and hasattr(self, "k_norm"):
                N = q.shape[0]
                q_flat = q.reshape(N * self.num_heads,
self.head_dim).contiguous()
                k_flat = k.reshape(N * self.num_kv_heads,
self.head_dim).contiguous()
                q = self.q_norm(q_flat).view(N, self.num_heads *
self.head_dim)
                k = self.k_norm(k_flat).view(N, self.num_kv_heads *
```

```
self.head_dim)
            attn_output = self.attn(q, k, v, input_metadata)
            output, _ = self.o_proj(attn_output)
            return output
```

Note: Make sure your `requirements.txt` includes all necessary dependencies. See the repository for the complete requirements list.

🎯 Quick Start

Launch SGLang server with TokenSelect.

Option 1: Using the provided script

```
bash scripts/serve.sh
```

Option 2: Manual command (example for Qwen2-7B-Instruct) Applied TokenSelect

```
python benchmark/serve.py \
    --model-path Qwen/Qwen2-7B-Instruct \
    --dp 1 \
    --disable-cuda-graph \
    --port 62726  \
    --mem-fraction-static 0.6 \
    --context-length 1048576 \
    --chunked-prefill-size 8192 \
    --max-prefill-tokens 1048576 \
    --sgl-conf-file config/qwen-token-retrieval.yaml
```

PROF

```
python benchmark/serve.py \
    --model-path Qwen/Qwen2-7B-Instruct \
    --dp 1 \
    --port 62726 \
    --disable-cuda-graph \
    --disable-regex-jump-forward \
    --disable-radix-cache \
    --max-running-requests 1 \
    --mem-fraction-static 0.85 \
    --context-length 1048576 \
    --sgl-conf-file config/qwen-token-retrieval.yaml
```

Option 3: Manual command (example for Qwen2-7B-Instruct) Applied SPDA

```
```

```
python benchmark/serve.py \
    --model-path Qwen/Qwen2-7B-Instruct \
    --dp 1 \
    --port 62726 \
    --disable-cuda-graph \
    --disable-regex-jump-forward \
    --use-spda \
    --disable-radix-cache \
    --max-running-requests 1 \
    --mem-fraction-static 0.85 \
    --context-length 1048576 \
    --sgl-conf-file config/qwen-token-retrieval.yaml
```

Send request to SGLang server using OpenAI Python Client. You can also use the benchmark/send_request.py script.

```
import openai

client = openai.Client(base_url=f"http://127.0.0.1:62726/v1",
api_key="None")

prompt = "The grass is green. The sky is blue. The sun is yellow. Here
we go. There and back again. " * 1000 + f"The pass key is Quang cười
haha. Remember it. Quang cười haha hihi is not the pass key. " + "The
grass is green. The sky is blue. The sun is yellow. Here we go. There
and back again. " * 1000 + "What is the pass key?"

response = client.chat.completions.create(
    model="Qwen/Qwen2-7B-Instruct",
    messages=[
        {"role": "user", "content": prompt},
    ],
    temperature=0,
)
print(response)
```

PROF

📊 Experiment

How to download all evaluation datas?

```
sudo apt update
sudo apt install aria2
bash scripts/download.sh
```

Evaluation on InfiniteBench

Download data from https://github.com/OpenBMB/Infini.

```
# using llama3
bash scripts/infinitebench-mp-llama.sh
# using qwen2
bash scripts/infinitebench-mp-qwen.sh
```

Evaluation on RULER

Download data from https://github.com/NVIDIA/RULER.

```
cd ruler
# using llama3
# bash run.sh model_name benchmark_name config_name port (choose an idle
port)
bash scripts/run.sh llama3-8b-inst synthetic llama-token-retrieval 63333
# using qwen2
# bash run.sh model_name benchmark_name config_name port (choose an idle
port)
bash scripts/run.sh qwen2-7b-inst synthetic qwen-token-retrieval 63333
```